Optimal Graph Burning: A Row Generation Approach for Large-Scale Network Contagion Modeling

Taha Zahid and Abdullah Shaikh
Habib University, Karachi
`{tz09220, as09245}@st.habib.edu.pk`

Abstract

This proposal outlines the implementation and evaluation of a Row Generation algorithm for the Graph Burning problem, as presented by Pereira et al. (2024) in ESA 2024. Graph burning is an NP-hard problem modeling the spread of influence in networks. We aim to implement the Integer Linear Programming (ILP) based row generation technique to find optimal burning sequences in large graphs, bridging the gap between theoretical combinatorial optimization and practical network analysis.

1 Introduction and Problem Selection

The chosen project follows **Option A: Paper-Based Implementation**. We have selected the paper *"A Row Generation Algorithm for Finding Optimal Burning Sequences of Large Graphs"* published in the **European Symposium on Algorithms (ESA 2024), Track B**.

The Graph Burning problem involves finding the minimum number of rounds, known as the burning number $b(G)$, to cover all nodes in a graph. In each round, a new fire source is placed on a node, and existing fires spread to their neighbors. This problem is relevant to modeling viral marketing, epidemic control, and social influence.

2 Brief Summary of Research Contribution

The primary contribution of the selected paper is a novel **Row Generation** (or lazy constraint) framework for solving the ILP formulation of graph burning. Standard ILP models for this problem suffer from a prohibitive number of constraints when applied to large graphs. (Pereira et al., 2024) propose starting with a relaxed model and iteratively adding coverage constraints only for nodes that remain unburned. This allows the solver to find optimal solutions for graphs with over 100,000 nodes, which was previously computationally infeasible.

2.1 Mathematical Formulation

The graph burning problem is modeled as a covering problem over k time steps. Let $G = (V, E)$ be a simple undirected graph. We define binary variables $x_{v,t} \in \{0, 1\}$ such that $x_{v,t} = 1$ if node v is chosen as a fire source at time $t \in \{1, \dots, k\}$. The objective is to find the minimum k such that:

$$\sum_{t=1}^{k} \sum_{v \in V} x_{v,t} = k \tag{1}$$

subject to the coverage constraint for every node $u \in V$:

$$\sum_{t=1}^{k} \sum_{v \in N_{k-t}(u)} x_{v,t} \geq 1 \tag{2}$$

where $N_d(u)$ is the set of vertices at a distance at most d from u.

3 Feasibility and Relevance Justification

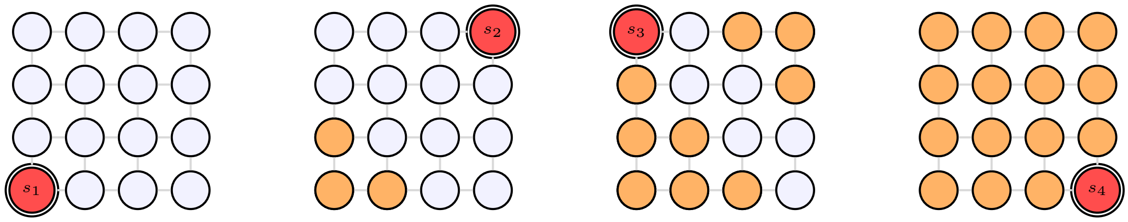
The Graph Burning problem is a canonical example of an NP-hard combinatorial optimization problem that defies standard exhaustive search methods.

The paper provides a structured, iterative framework (Row Generation) that can be built incrementally. By leveraging modern optimization libraries such as `PuLP` or `PySCIPOpt`, we can focus on the high-level algorithmic logic rather than the low-level mechanics of the ILP solver.

4 Implementation and Experiment Plan

The implementation will be divided into three distinct phases to ensure a modular and verifiable development process:

1. **Heuristic Baselining:** We will implement the *Modified GrOpt* (Greedy Optimization) heuristic described in the paper to provide an initial upper bound for the burning number k. This serves as the primary benchmark for our optimal solver.



Round 1: s_1 **Round 2**: s_2 **Round 3**: s_3 **Round 4**: s_4 (Optimal)

Figure 1: Step-by-step burning on a 4×4 grid. In each round i, we add a source s_i (red) and existing fires expand to distance 1 (orange). By $t = 4$, the entire grid is covered, thus $b(G) = 4$.

2. **Core Row Generation Loop:** We will develop a solver that initializes with a relaxed ILP model. A Breadth-First Search (BFS) based separation algorithm will be implemented to identify unburned nodes and dynamically inject "lazy constraints" (rows) into the model until an optimal k-sequence is found.

3. **Graph Topology Suite:** To evaluate performance, we will generate a synthetic dataset including Paths, Cycles, Grids, and Random Graphs (Erdős-Rényi), as well as utilize real-world network datasets from the SNAP library to test scalability.

Graph Topology	**Source/Type**	**Node Count** ($\lvert V \rvert$)
Path & Cycle Graphs	Synthetic	$10^2 - 10^5$
Grid/Mesh Networks	Synthetic	100×100
Social Circles	SNAP (Facebook)	$4,039$
P2P Network	SNAP (Gnutella)	$6,301$
Web Graph	SNAP (Google)	$875,713$

Table 1: Proposed datasets for scalability and performance benchmarking. Real-world datasets are sourced from the SNAP library.

Algorithm 1: Row Generation for GBP

Input: Graph $G = (V, E)$, candidate burning number k
1. Initialization: Create ILP model with objective $\sum x_{v,t} = k$ and no coverage constraints.
2. Solve: Find a solution S (set of k fire sources).
3. Separate:
 a. Perform multi-source BFS from S to calculate coverage at time k.
 b. Identify the set of unburned nodes $U \subset V$.
4. Check:
 If $U = \emptyset$: **Return** Optimal solution found.
 Else: Select node $u \in U$, add its coverage constraint to ILP, and **GOTO 2**.

4.1 Algorithmic Approach

The core of our implementation is the **Separation Oracle**. Instead of loading all $|V|$ constraints into the ILP solver, we follow the iterative process outlined in Algorithm 1. This "lazy" approach ensures that we only consider the critical constraints necessary to prove optimality for a given burning number k.

4.2 Evaluation Metrics

To ensure a rigorous evaluation, we will measure:

1. **Optimality Gap:** Comparing the results of our Row Generation algorithm against the Greedy heuristic baseline.

2. **Computational Efficiency:** Measuring the execution time and the number of constraints (rows) actually generated versus the total number of nodes.

3. **Scalability:** Testing the algorithm on graphs of increasing order (n) to identify the practical breaking point of the implementation.

4.3 Anticipated Technical Challenges

While the row generation approach is theoretically robust, we anticipate several engineering challenges during the implementation phase:

- **Memory-Efficient Distance Checks:** For the largest graphs (e.g., the Google web graph with $n \approx 875k$), storing an all-pairs shortest path matrix is infeasible ($O(n^2)$ space). We will mitigate this by implementing an "on-the-fly" BFS within our separation oracle to check coverage only for the current candidate source set.

- **Heuristic Accuracy:** If the initial *Modified GrOpt* heuristic provides a poor upper bound, the ILP may require an excessive number of iterations. To address this, we will implement the *Priority-based Row Selection* strategy mentioned in the paper to prioritize "influential" nodes that cover high-degree neighborhoods.

- **Solver Convergence:** ILP solvers can sometimes struggle with the symmetry inherent in graph structures. We will utilize the symmetry-breaking constraints discussed in Section 4.2 of the paper to ensure the solver converges within a reasonable time-frame for each round k.

5 Resources Declaration

We will use Python as the primary language. Core libraries include NetworkX for graph theory operations, PuLP for ILP modeling, and Matplotlib for data visualization. Experiments will be conducted on personal workstations with standard CPU configurations.

References

Felipe de Carvalho Pereira, Pedro Jussieu de Rezende, Tallys Yunes, and Luiz Fernando Batista Morato. 2024. A Row Generation Algorithm for Finding Optimal Burning Sequences of Large Graphs. In *32nd Annual European Symposium on Algorithms (ESA 2024)*, volume 308 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 94:1–94:17, Dagstuhl, Germany. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.